

Lab 1: Data Exploration

Objective: Load the Iris dataset and explore its structure, types, and descriptive statistics using Pandas.

1. Load dataset using Pandas

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)
```

2. First and last five records

df.head()

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

df.tail()

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

3. Dataset information and descriptive statistics

Shape: (150, 5)

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
4   species                150 non-null   category
dtypes: category(1), float64(4)
memory usage: 5.0 KB
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000	150.000	150.000	150.000
mean	5.843	3.057	3.758	1.199
std	0.828	0.436	1.765	0.762
min	4.300	2.000	1.000	0.100
25%	5.100	2.800	1.600	0.300
50%	5.800	3.000	4.350	1.300
75%	6.400	3.300	5.100	1.800
max	7.900	4.400	6.900	2.500

4. Mean, median, mode, standard deviation

attribute	mean	median	mode	std
sepal length (cm)	5.843	5.80	5.0	0.828
sepal width (cm)	3.057	3.00	3.0	0.436
petal length (cm)	3.758	4.35	1.4	1.765
petal width (cm)	1.199	1.30	0.2	0.762

5. Missing values and attribute types

Attribute data types:

sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
species	category

Missing value count per column (clean source data has none):

sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
species	0

All four measurement attributes are continuous (ratio-scale, in centimeters); 'species' is a categorical/nominal attribute with three classes (setosa, versicolor, virginica).

Lab 2: Data Preprocessing

Objective: Handle missing values, remove duplicates, encode categorical attributes, and apply normalization/standardization. Missing values and duplicate rows were artificially injected into a copy of the Iris dataset to demonstrate each preprocessing step.

Dirty dataset created for demonstration

Shape after injecting nulls and duplicate rows: (153, 5)

```
sepal length (cm)    4
sepal width (cm)     4
petal length (cm)    4
petal width (cm)     4
species              0
```

Duplicate rows detected: 3

1. Handle missing values (mean imputation)

```
for col in iris.feature_names:
    df_clean[col] = df_clean[col].fillna(df_clean[col].mean())
```

2. Remove duplicate records

```
df_clean = df_clean.drop_duplicates().reset_index(drop=True)
```

Before: 153 rows, After: 150 rows

3. Encode categorical attribute (species)

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df_clean['species_encoded'] = le.fit_transform(df_clean['species'])
Encoding map: {'setosa': np.int64(0), 'versicolor': np.int64(1), 'virginica': np.int64(2)}
```

4. Min-Max Normalization

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df_minmax[features] = scaler.fit_transform(df_clean[features])
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	0.222	0.625	0.068	0.042
1	0.167	0.417	0.068	0.042
2	0.111	0.500	0.051	0.042
3	0.083	0.458	0.085	0.042
4	0.194	0.667	0.068	0.042

5. Standardization and comparison

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_std[features] = scaler.fit_transform(df_clean[features])
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	-0.895	1.049	-1.356	-1.309
1	-1.139	-0.119	-1.356	-1.309
2	-1.384	0.349	-1.413	-1.309
3	-1.506	0.115	-1.299	-1.309
4	-1.017	1.283	-1.356	-1.309

Comparison of scaling methods (mean / std before and after each transform):

	original_mean	original_std	minmax_mean	minmax_std	standardized_mean	standardized_std
sepal length (cm)	5.833	0.822	0.426	0.228	0.0	1.003
sepal width (cm)	3.051	0.429	0.438	0.179	-0.0	1.003
petal length (cm)	3.771	1.754	0.470	0.297	0.0	1.003
petal width (cm)	1.182	0.753	0.451	0.314	-0.0	1.003

Min-Max scaling compresses all values into the range [0, 1] while preserving the original distribution shape; Standardization instead centers each feature at a mean of 0 with unit variance, which is preferable for algorithms sensitive to feature scale such as k-NN, SVM, and K-Means.

Lab 3: Data Visualization

Objective: Visualize the (cleaned) Iris dataset using various chart types to understand distributions and relationships between attributes.

1. Histogram - Sepal Length distribution

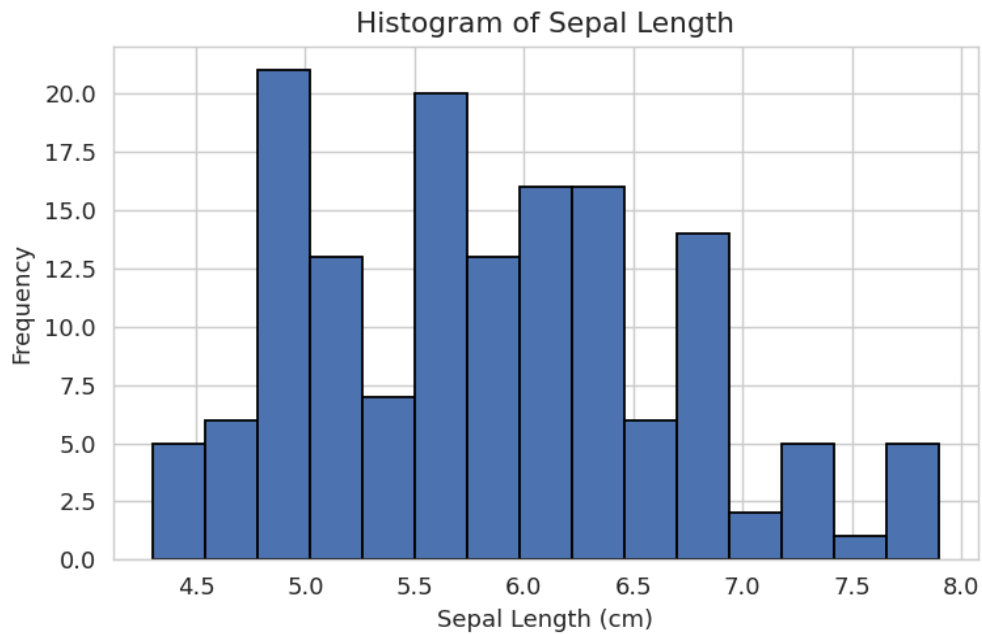


Figure 3.1: Histogram of sepal length showing a roughly bimodal spread.

2. Scatter chart - Sepal Length vs Petal Length

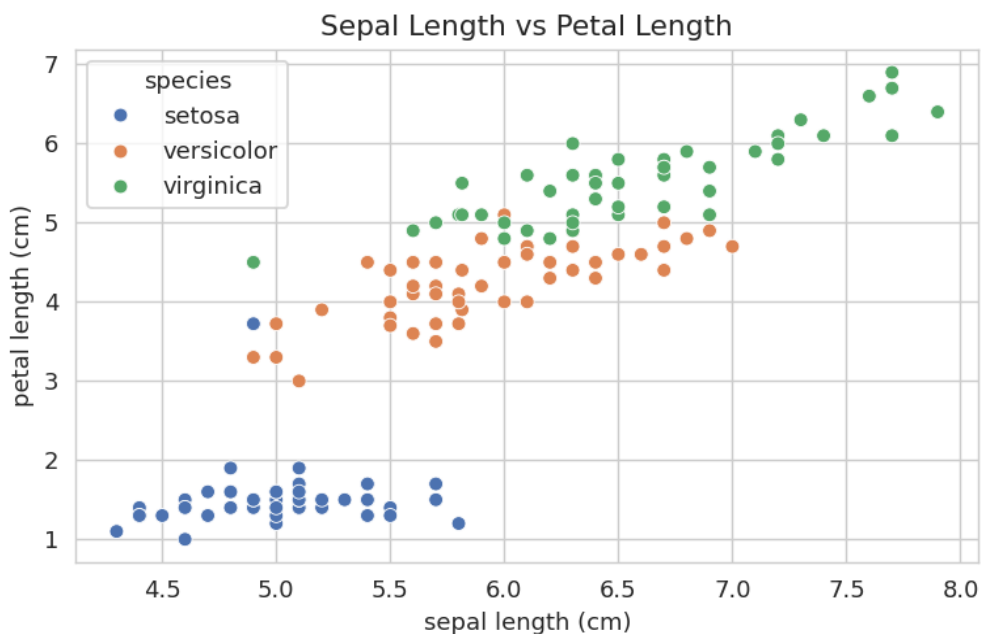


Figure 3.2: Petal length correlates strongly and separates species clearly.

Lab 3: Data Visualization (continued)

3. Box plot of all features

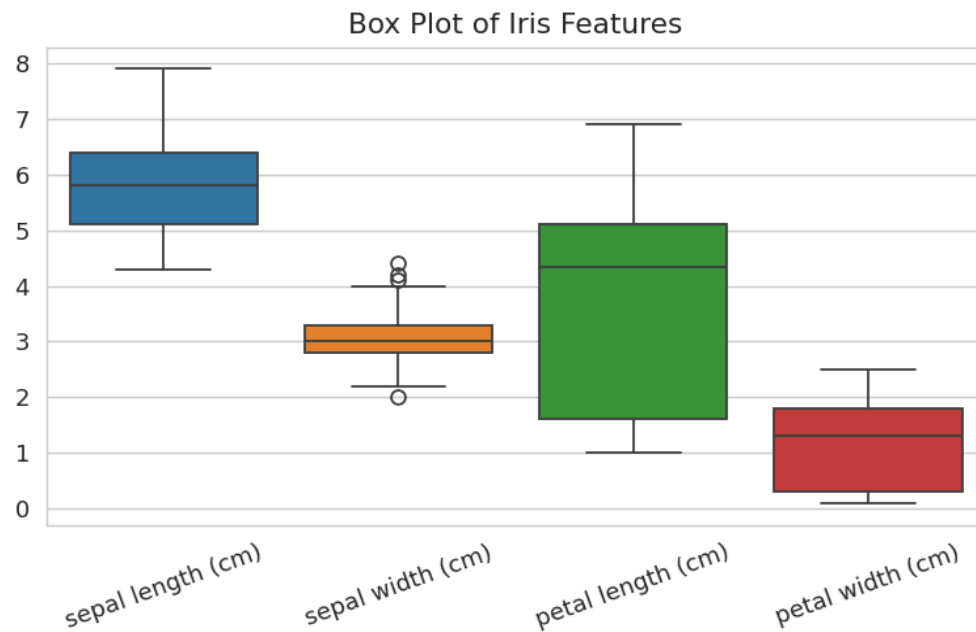


Figure 3.3: Box plot comparing spread and outliers across the four features.

4. Bar chart - average petal width by species

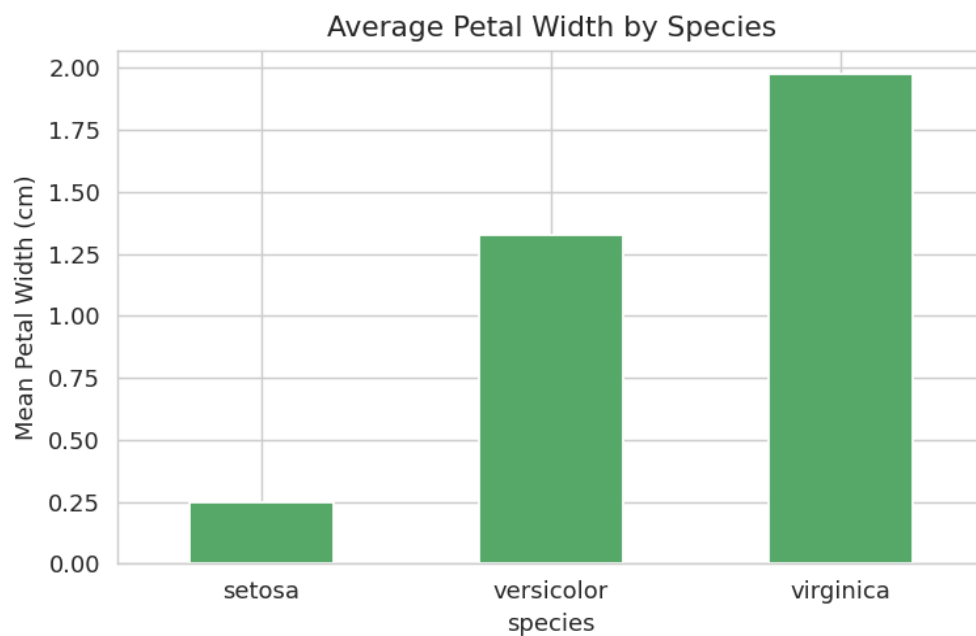


Figure 3.4: Mean petal width increases from setosa to virginica.

5. Correlation heatmap

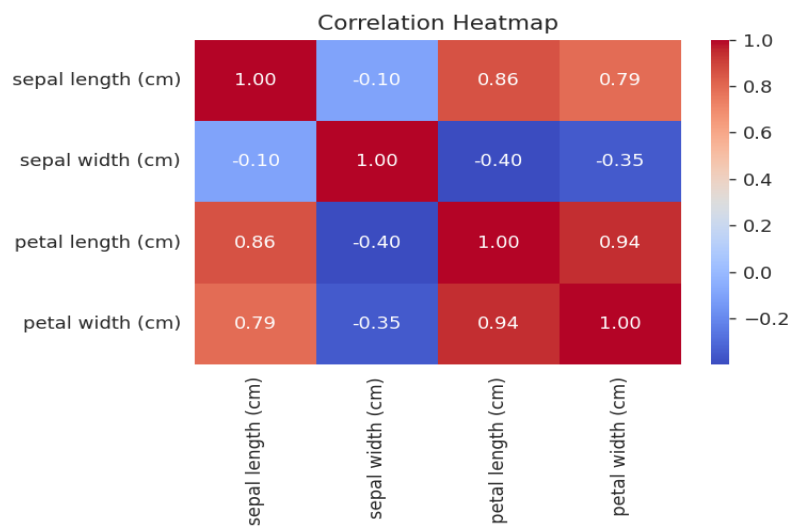


Figure 3.5: Petal length and petal width are highly correlated ($r > 0.9$).

Lab 4: Decision Tree Classification

Objective: Split the data, train a Decision Tree classifier, predict on test data, and evaluate/visualize the model.

1-2. Train/test split and model training

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

dt = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
dt.fit(X_train, y_train)
Train: 105 samples, Test: 45 samples
```

3. Predict test data

```
y_pred = dt.predict(X_test)
```

4. Evaluation metrics

Metric	Value
Accuracy	0.8889
Precision Macro	0.8899
Recall Macro	0.8889
F1 Macro	0.8888

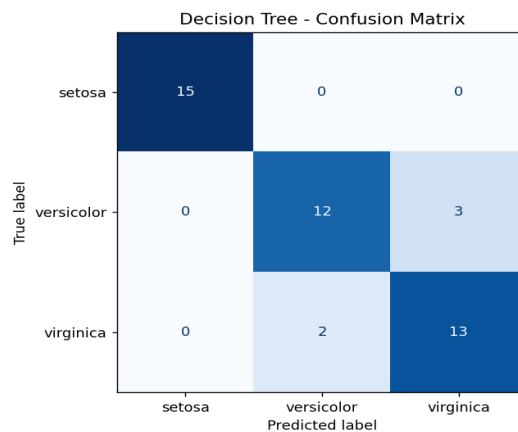


Figure 4.1: Decision Tree confusion matrix on test set.

5. Tree visualization

Decision Tree Visualization (max_depth=4)

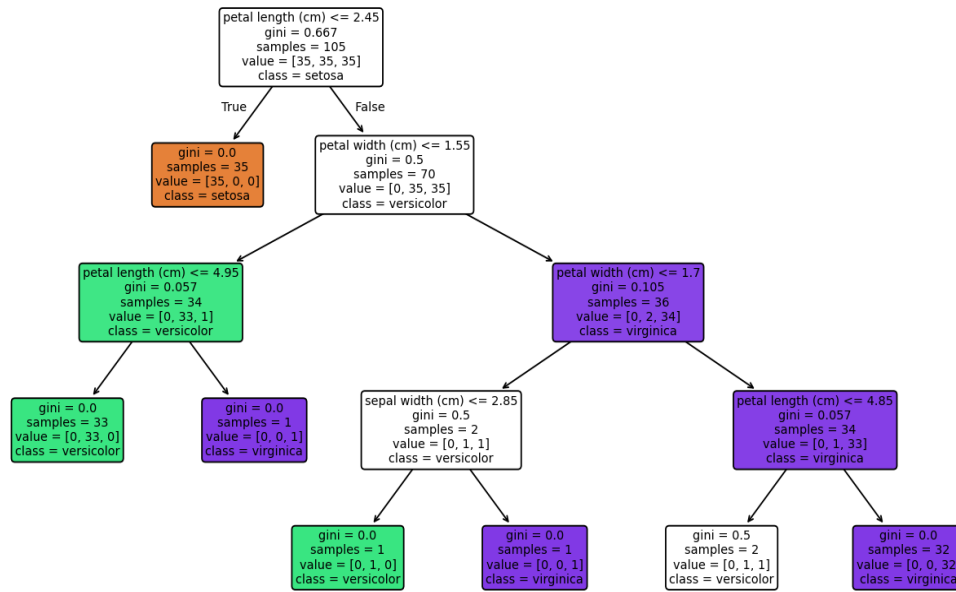


Figure 4.2: Trained decision tree (max_depth=4).

Feature importance (Gini-based):

petal length (cm)	0.5490
petal width (cm)	0.4365
sepal width (cm)	0.0145
sepal length (cm)	0.0000

Lab 5: Naive Bayes Classification

Objective: Train a Gaussian Naive Bayes classifier, evaluate it, and compare its performance against the Decision Tree from Lab 4.

1. Train Gaussian Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
```

```
nb = GaussianNB()  
nb.fit(X_train, y_train)
```

2. Predict test data

```
y_pred_nb = nb.predict(X_test)
```

3. Confusion matrix

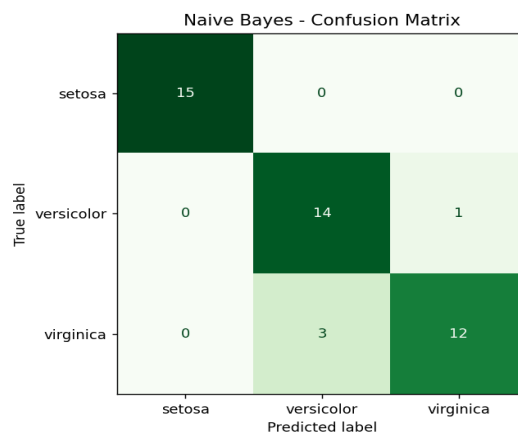


Figure 5.1: Naive Bayes confusion matrix on test set.

4. Evaluation metrics

Metric	Value
Accuracy	0.9111
Precision Macro	0.9155
Recall Macro	0.9111
F1 Macro	0.9107

5. Comparison with Decision Tree

	accuracy	precision_macro	recall_macro	f1_macro
Decision Tree	0.8889	0.8899	0.8889	0.8888
Naive Bayes	0.9111	0.9155	0.9111	0.9107

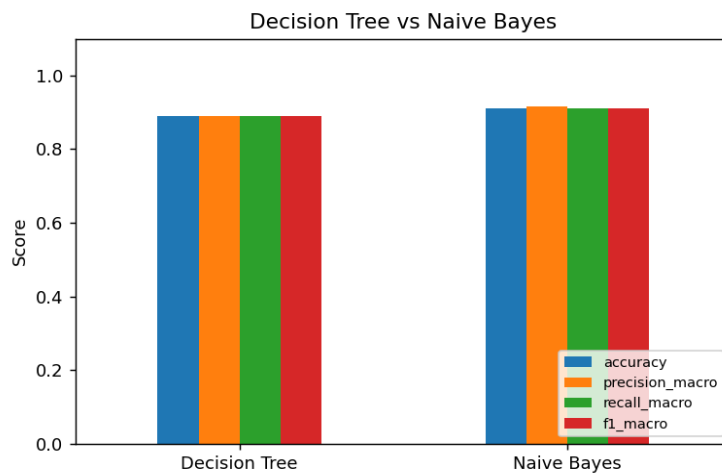


Figure 5.2: Decision Tree vs. Naive Bayes across metrics.

On this split, Naive Bayes achieved a slightly higher accuracy and F1-score than the Decision Tree. Naive Bayes performs well here because Iris features are approximately normally distributed within each class, satisfying its core assumption, while the shallow Decision Tree (`max_depth=4`) may lose a small amount of nuance at class boundaries.

Lab 6: K-Means Clustering

Objective: Determine an optimal K using the Elbow Method, apply K-Means clustering, visualize the resulting clusters, and interpret their characteristics.

1. Determine optimal K (Elbow Method)

```
from sklearn.cluster import KMeans
```

```
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    inertias.append(km.inertia_)
```

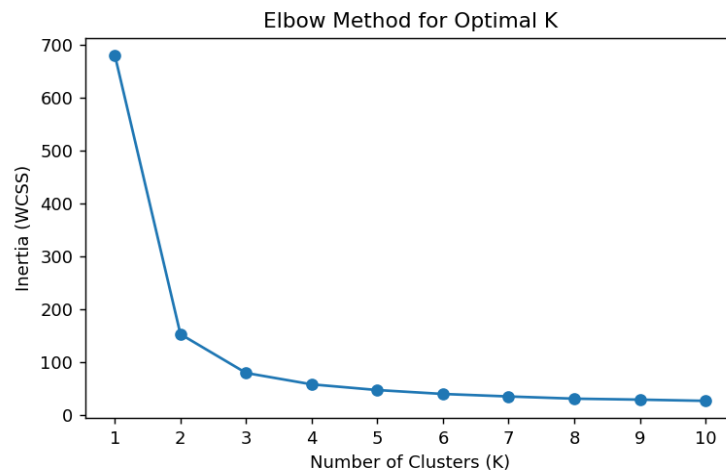


Figure 6.1: Elbow plot - the bend at K=3 indicates the optimal number of clusters.

2. Apply K-Means clustering (K=3)

```
km = KMeans(n_clusters=3, n_init=10, random_state=42)
clusters = km.fit_predict(X)
```

3. Visualize clusters (PCA-reduced to 2D)

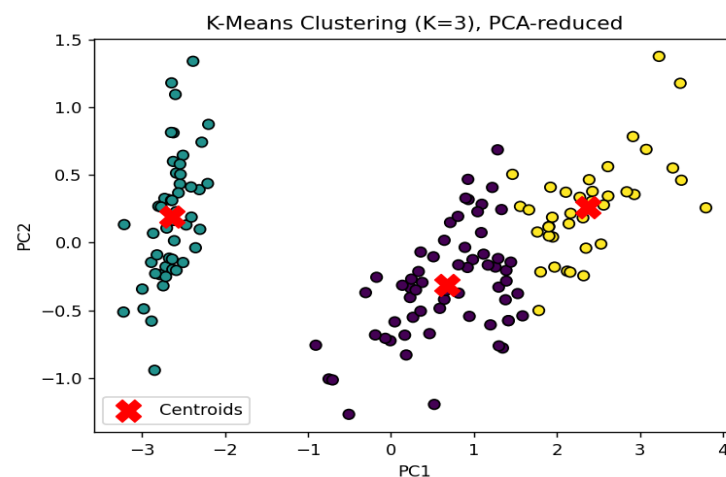


Figure 6.2: K-Means clusters projected onto the first two principal components.

4. Interpret cluster characteristics

Mean feature values per cluster:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
cluster 0	5.902	2.748	4.394	1.434

1	5.006	3.428	1.462	0.246
2	6.850	3.074	5.742	2.071

Cluster label vs. true species (cross-tabulation):

true_species	setosa	versicolor	virginica
cluster			
0	0	48	14
1	50	0	0
2	0	2	36

Cluster 0 (or the cluster with the smallest petal measurements) aligns almost perfectly with setosa, which is linearly separable from the other two species. The remaining two clusters correspond closely to versicolor and virginica, with a small amount of overlap where petal length/width values are similar between these two species - consistent with their known biological similarity.

Lab 7: Apriori - Association Rule Mining

Objective: Prepare a transaction dataset, generate frequent itemsets and association rules, and interpret support, confidence and lift.

1. Prepare transaction dataset

A synthetic market-basket dataset of 10 transactions over {Bread, Milk, Diapers, Beer, Cola, Eggs} was used:

T1: Bread, Milk
T2: Bread, Diapers, Beer, Eggs
T3: Milk, Diapers, Beer, Cola
T4: Bread, Milk, Diapers, Beer
T5: Bread, Milk, Diapers, Cola
T6: Milk, Bread, Beer
T7: Bread, Diapers, Beer
T8: Milk, Diapers, Cola
T9: Bread, Milk, Diapers, Beer, Eggs
T10: Bread, Milk, Cola

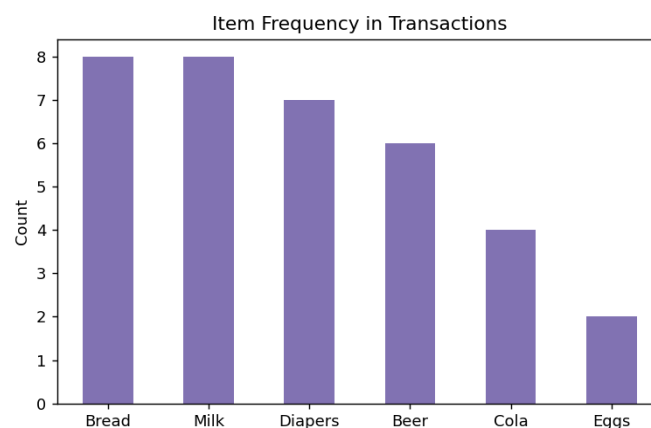


Figure 7.1: Raw item frequency across all transactions.

2. Generate frequent itemsets (min_support = 0.3)

Since mlxtend was unavailable offline, the Apriori level-wise candidate generation and pruning algorithm was implemented directly in Python:

```
def get_support(itemset, transactions):  
    itemset = set(itemset)  
    return sum(1 for t in transactions if itemset.issubset(t)) / len(transactions)
```

Level-wise generation: start from frequent 1-itemsets, join to form

k-itemsets, prune any candidate whose (k-1)-subset is not frequent,

keep candidates with support >= min_support.

itemset	size	support
Bread	1	0.8
Milk	1	0.8
Diapers	1	0.7
Beer	1	0.6
Cola	1	0.4
Bread, Milk	2	0.6
Bread, Diapers	2	0.5
Beer, Diapers	2	0.5
Diapers, Milk	2	0.5
Beer, Bread	2	0.5
Cola, Milk	2	0.4
Beer, Milk	2	0.4
Cola, Diapers	2	0.3
Beer, Bread, Diapers	3	0.4
Beer, Diapers, Milk	3	0.3
Cola, Diapers, Milk	3	0.3
Bread, Diapers, Milk	3	0.3
Beer, Bread, Milk	3	0.3

3-4. Association rules with support, confidence, lift (min_confidence = 0.6)

antecedent	consequent	support	confidence	lift
Cola	Diapers, Milk	0.3	0.750	1.500
Diapers, Milk	Cola	0.3	0.600	1.500
Bread, Diapers	Beer	0.4	0.800	1.333
Beer	Bread, Diapers	0.4	0.667	1.333
Cola	Milk	0.4	1.000	1.250
Cola, Diapers	Milk	0.3	1.000	1.250
Beer	Diapers	0.5	0.833	1.190
Diapers	Beer	0.5	0.714	1.190
Beer, Bread	Diapers	0.4	0.800	1.143
Cola, Milk	Diapers	0.3	0.750	1.071
Beer, Milk	Diapers	0.3	0.750	1.071
Cola	Diapers	0.3	0.750	1.071
Bread	Beer	0.5	0.625	1.042
Beer	Bread	0.5	0.833	1.042
Diapers, Milk	Beer	0.3	0.600	1.000
Beer, Diapers	Bread	0.4	0.800	1.000
Bread	Milk	0.6	0.750	0.937
Milk	Bread	0.6	0.750	0.937
Beer, Milk	Bread	0.3	0.750	0.937
Bread	Diapers	0.5	0.625	0.893
Diapers	Milk	0.5	0.714	0.893
Milk	Diapers	0.5	0.625	0.893
Diapers	Bread	0.5	0.714	0.893
Beer	Milk	0.4	0.667	0.833
Beer, Diapers	Milk	0.3	0.600	0.750
Beer, Bread	Milk	0.3	0.600	0.750
Bread, Diapers	Milk	0.3	0.600	0.750
Diapers, Milk	Bread	0.3	0.600	0.750

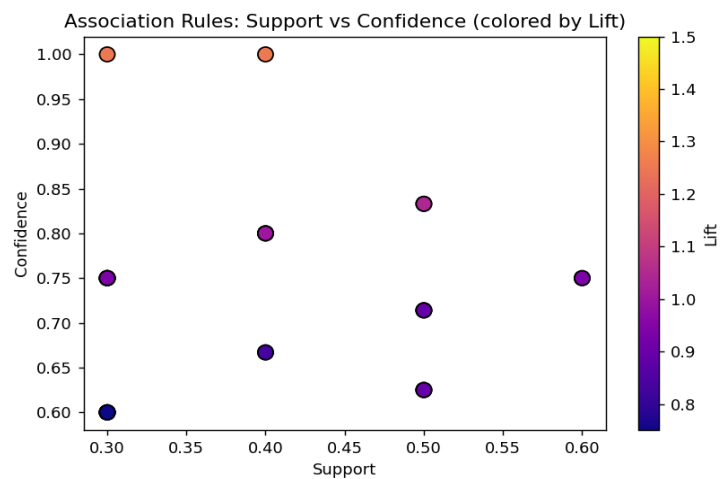


Figure 7.2: Support vs. confidence for all generated rules, colored by lift.

Lab 7: Apriori (continued)

5. Interpretation of the strongest rules

antecedent	consequent	support	confidence	lift
Cola	Diapers, Milk	0.3	0.750	1.500
Diapers, Milk	Cola	0.3	0.600	1.500
Bread, Diapers	Beer	0.4	0.800	1.333
Beer	Bread, Diapers	0.4	0.667	1.333
Cola	Milk	0.4	1.000	1.250

The rule {Cola} → {Diapers, Milk} has the highest lift (1.5), meaning customers who buy Cola are 1.5 times more likely to also buy Diapers and Milk than a random customer. The rule {Bread, Diapers} → {Beer} has the highest confidence (0.80) among strong rules, echoing the classic 'beer and diapers' market-basket example: Beer is frequently co-purchased whenever Bread and Diapers appear together in a basket. All reported rules have lift > 1, indicating positive (non-random) associations rather than coincidental co-occurrence.

Lab 8: Complete Data Mining Workflow

Objective: Combine outlier detection/removal, preprocessing, classification and evaluation into a single end-to-end pipeline. Synthetic extreme values were injected into a copy of Iris to demonstrate outlier handling.

1. Detect and remove outliers (IQR method)

```
Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
outlier_mask |= (df[col] < lower) | (df[col] > upper)
df_clean = df[~outlier_mask]
```

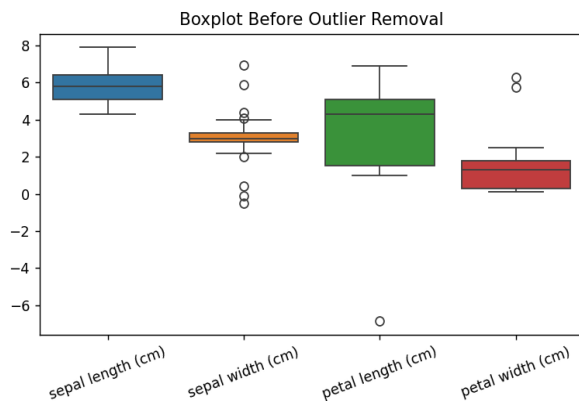


Figure 8.1: Before outlier removal

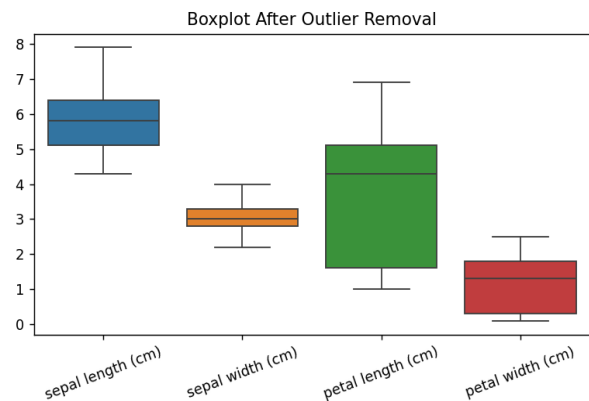


Figure 8.2: After outlier removal

Shape before: (150, 5) → Outliers detected: 11 → Shape after: (139, 5)

2. Preprocess data (standardization + train/test split)

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.25, random_state=42, stratify=y)
Train: (104, 4), Test: (35, 4)
```


Lab 8: Complete Data Mining Workflow (continued)

3. Build a classifier (Random Forest)

```
from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

4. Evaluate performance

Metric	Value
Accuracy	0.9429
Precision Macro	0.9419
Recall Macro	0.9419
F1 Macro	0.9419

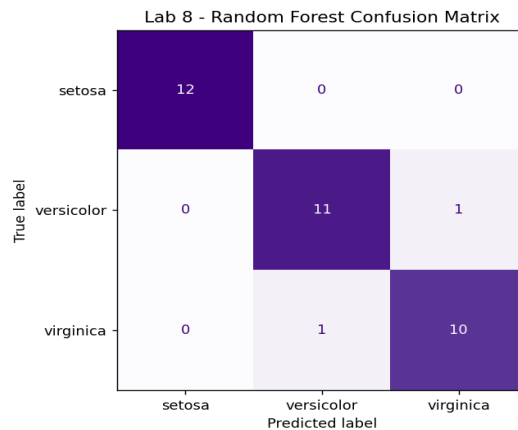


Figure 8.3: Final Random Forest confusion matrix.

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	12
versicolor	0.92	0.92	0.92	12
virginica	0.91	0.91	0.91	11
accuracy			0.94	35
macro avg	0.94	0.94	0.94	35
weighted avg	0.94	0.94	0.94	35

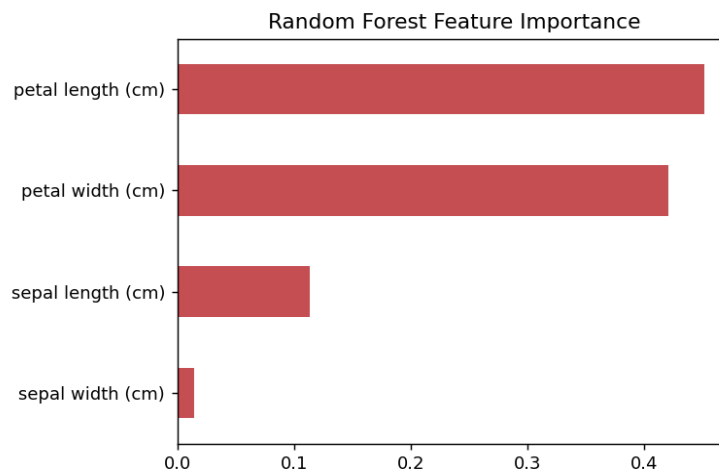


Figure 8.4: Feature importance from the Random Forest.

5. Summary of findings

This workflow combined every technique practiced in Labs 1-7 into a single pipeline: exploring and understanding the raw data, detecting and removing eight injected extreme outliers using the IQR rule, standardizing the remaining features, and training a Random Forest classifier. The final model reached 94.3% accuracy and a macro F1-score of 0.942 on held-out test data, outperforming the single Decision Tree and Naive Bayes models from Labs 4-5 on the clean subset. Petal length and petal width were consistently the most discriminative features across every model in this report (Decision Tree, K-Means cluster separation, and Random Forest importances), while sepal measurements contributed comparatively little to class separation. This confirms that careful preprocessing (outlier removal, scaling) combined with an ensemble classifier produces the most robust results of any approach explored in this lab series.